TEXAS A&M UNIVERSITY-SAN ANTONIO

CSCI 2436 Data Structures

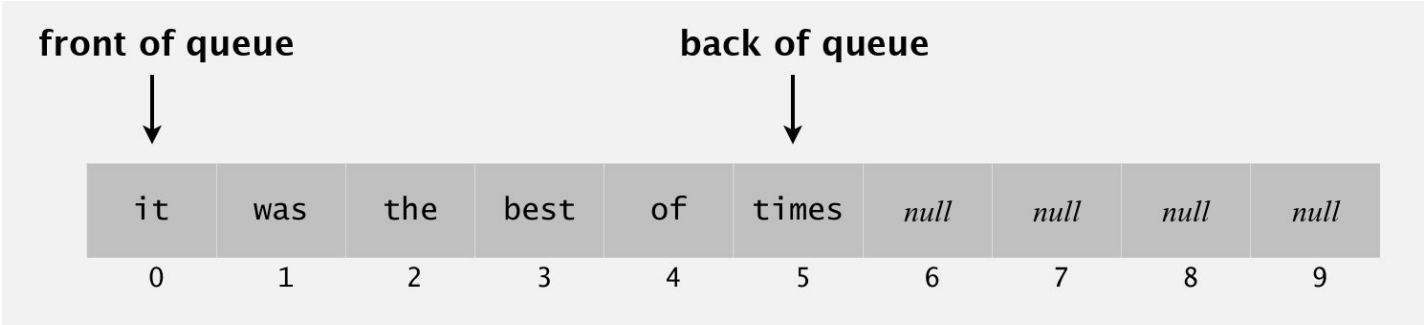
Spring 2026

Chapter 8 Queues, Deque and
Priority Queue Implementations

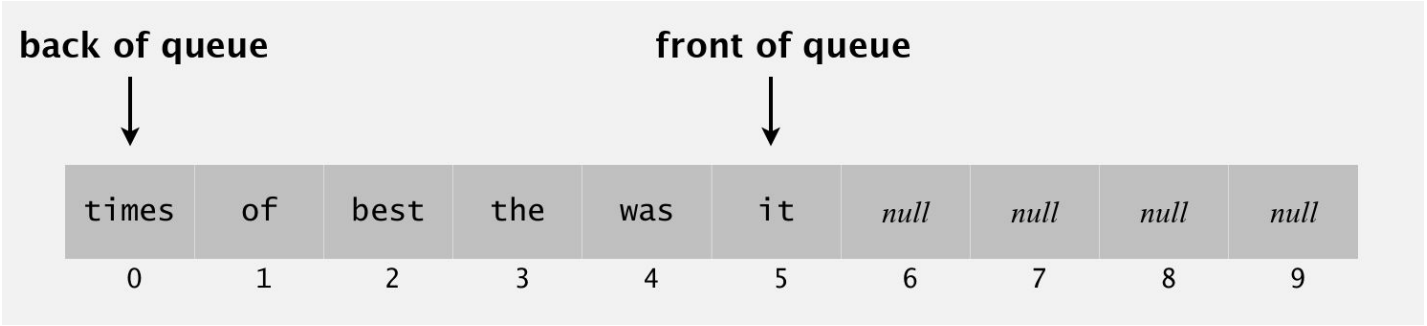
Array-Based Implementation of a Queue

- Enqueue "*it was the best of times*" to a queue, where are the front and back of the queue? How do you like them to be stored?

A

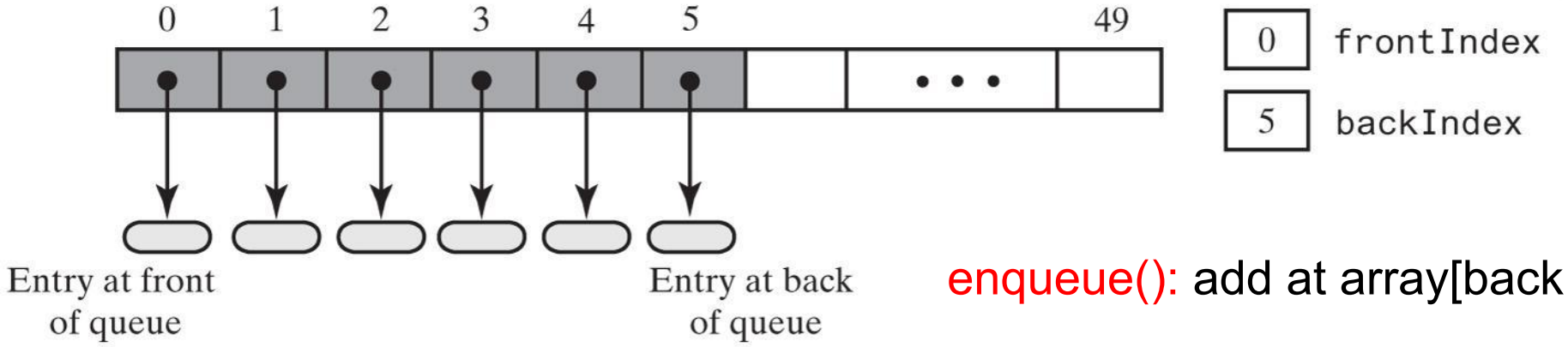


B



Array-Based Implementation of a Queue

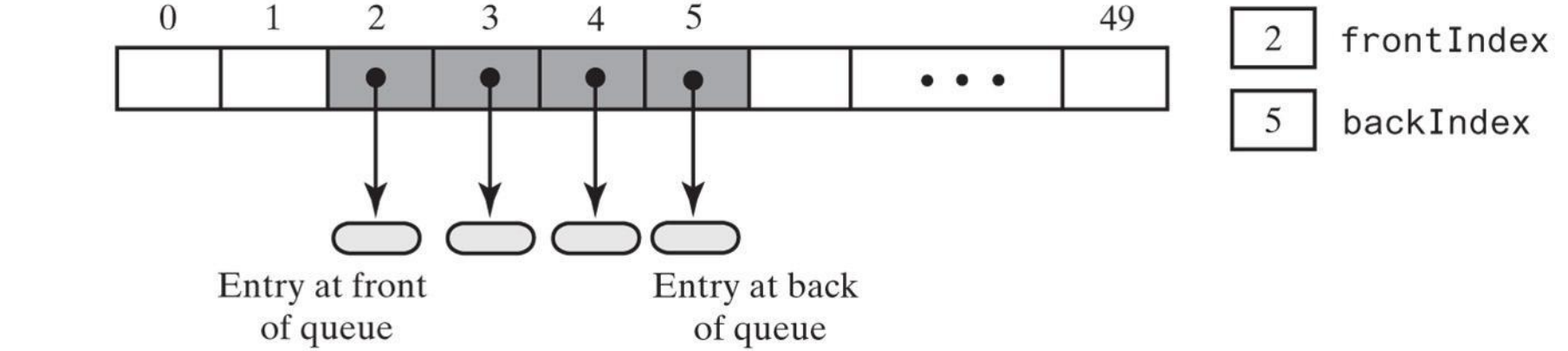
(a) The queue initially



enqueue(): add at array[backIndex +1]

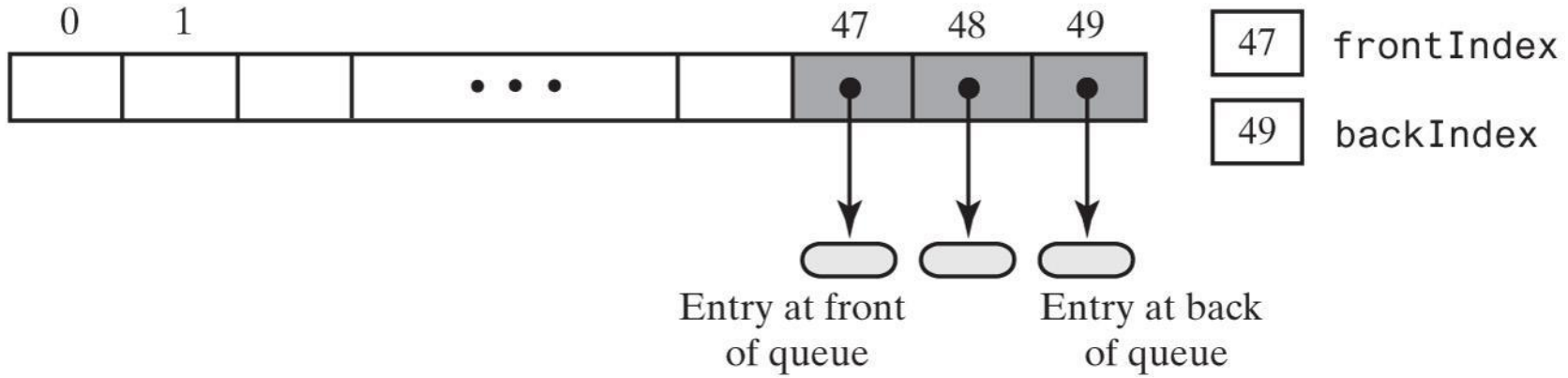
dequeue(): remove from array[frontIndex]

(b) After two removals of the front entry



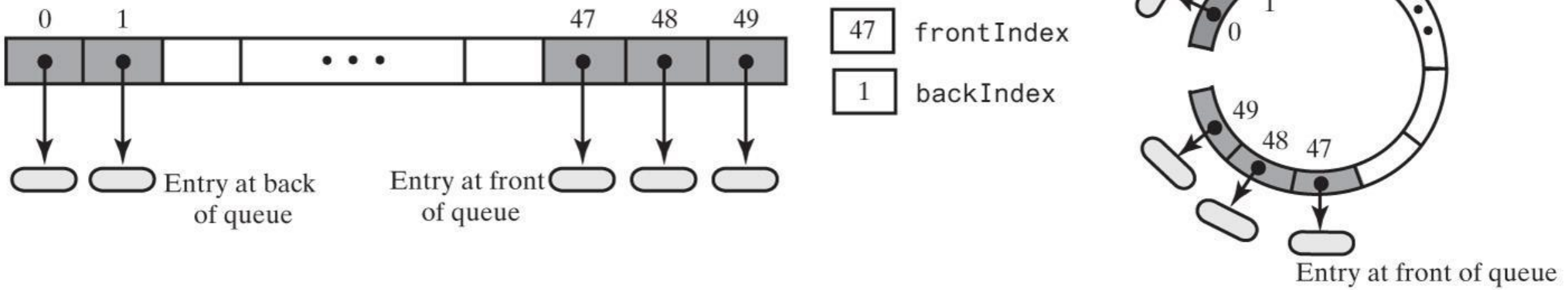
Array-Based Implementation of a Queue

(c) After several more additions and removals



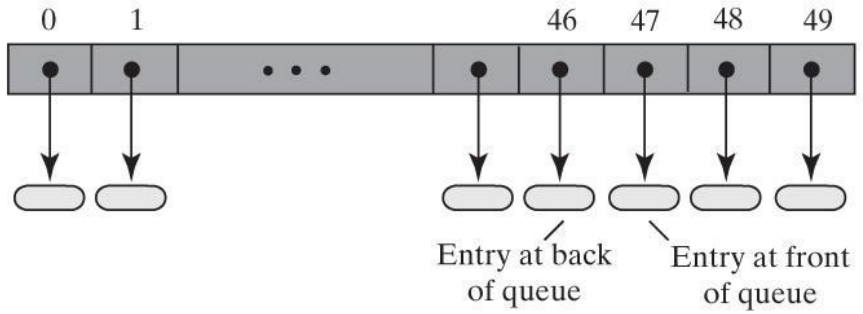
Oops…

Circular Array



Circular Array

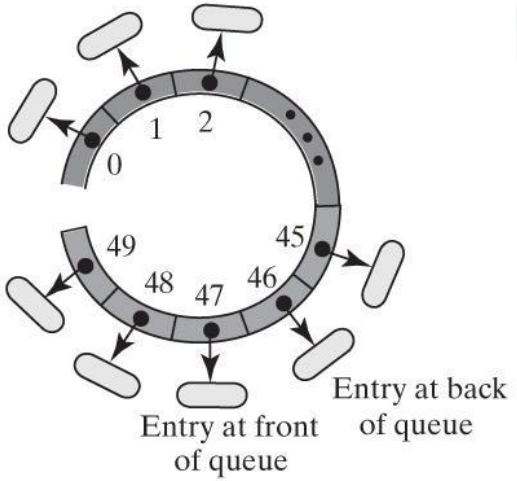
(a) After adding more entries to the queue in Figure 8-7 until it is full



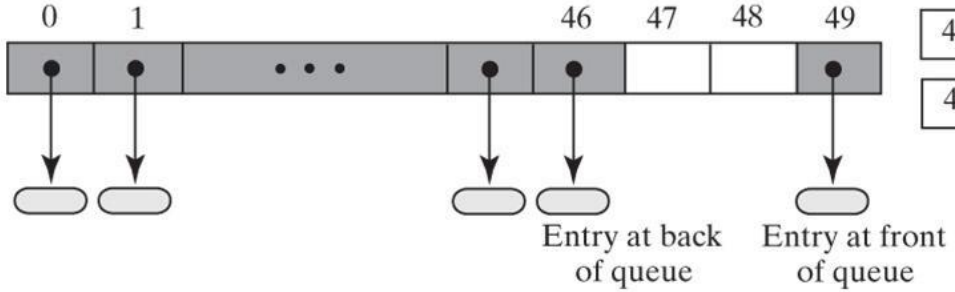
47 frontIndex

46 backIndex

Full

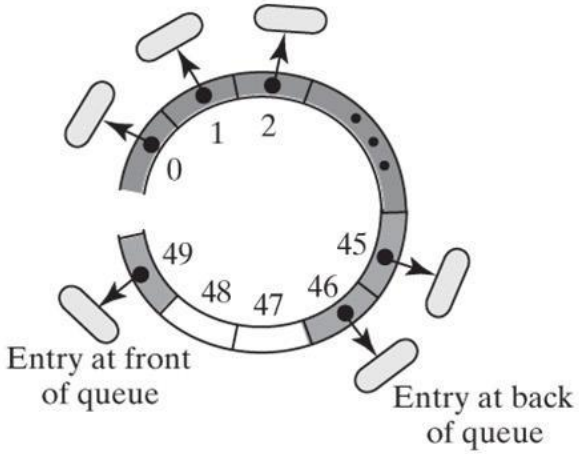


(b) After removing two entries



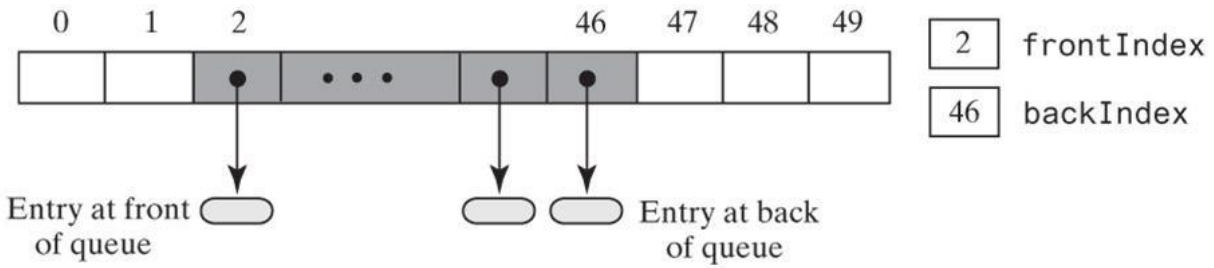
49 frontIndex

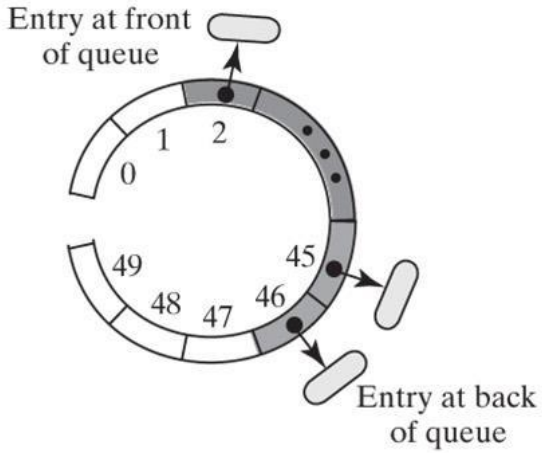
46 backIndex



Circular Array

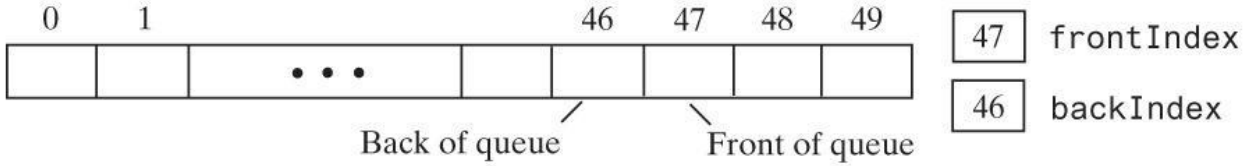
(c) After removing three more entries



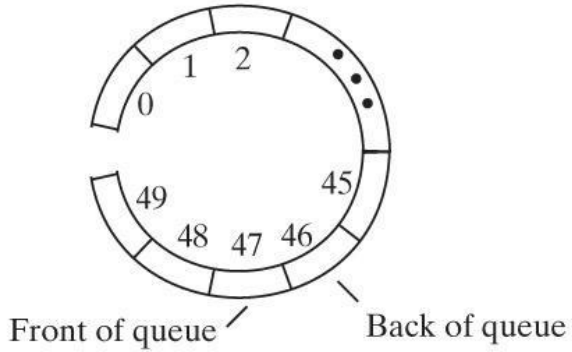


How can we test if a queue is full or empty?

(e) After removing the remaining entry, making the queue empty

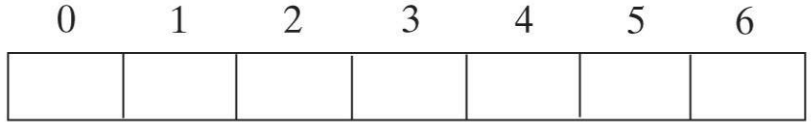


Empty



Circular Array with One Unused Element

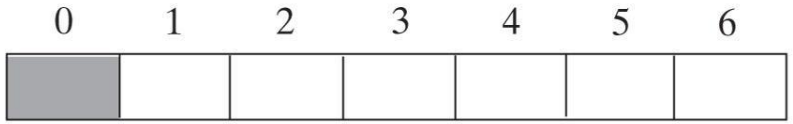
(a) Initially, the queue is empty



0 frontIndex

6 backIndex

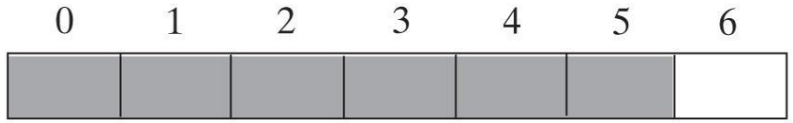
(b) After enqueuing one entry



0 frontIndex

0 backIndex

(c) After enqueuing five more entries, the queue is full

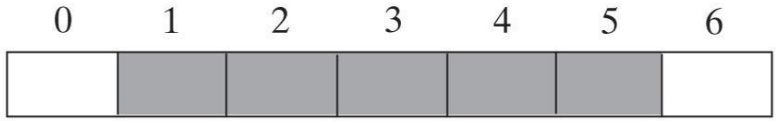


0 frontIndex

5 backIndex

Full

(d) After dequeuing an entry



1 frontIndex

5 backIndex

Circular Array with One Unused Element

(e) After enqueuing an entry, the queue becomes full again

1 frontIndex

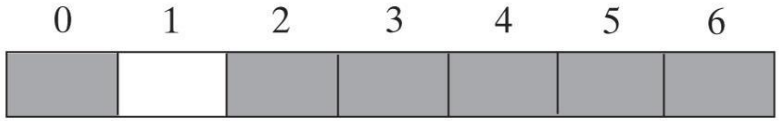
6 backIndex

(f) After dequeuing an entry

2 frontIndex

6 backIndex

(g) After enqueuing an entry, the queue is full

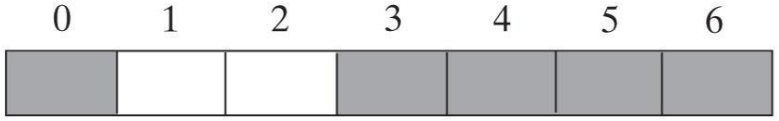


2 frontIndex

0 backIndex

Full

(h) After dequeuing an entry

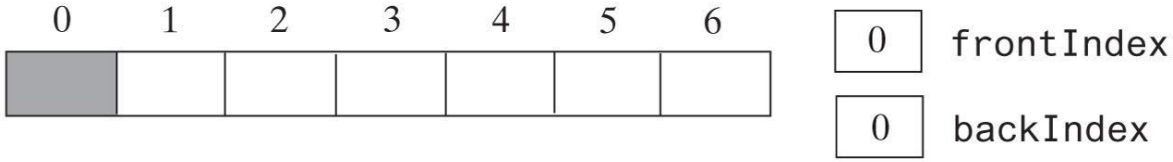


3 frontIndex

0 backIndex

Circular Array with One Unused Element

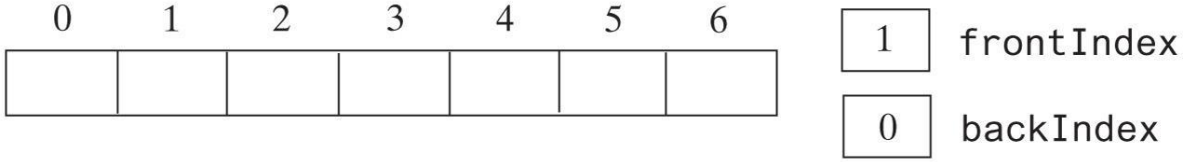
(i) After dequeuing all but one entry



How to test full vs empty by using only frontIndex and backIndex?

(j) After dequeuing the remaining entry, the queue is now empty

Empty

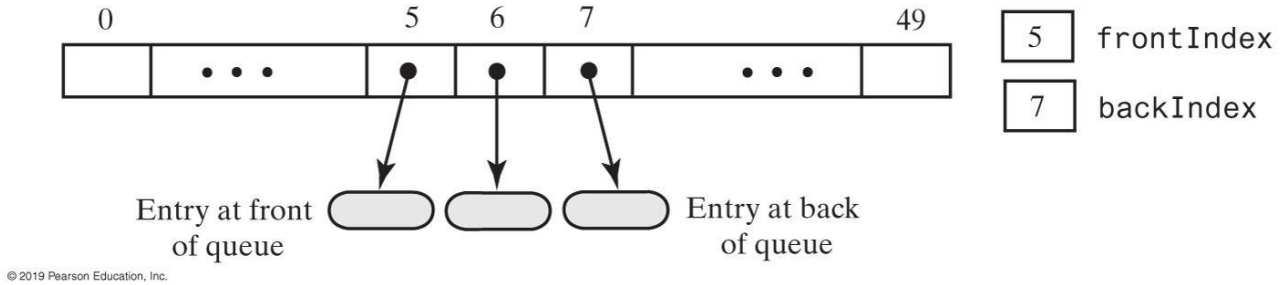


Empty: frontIndex = (backIndex + 1) % queue.length

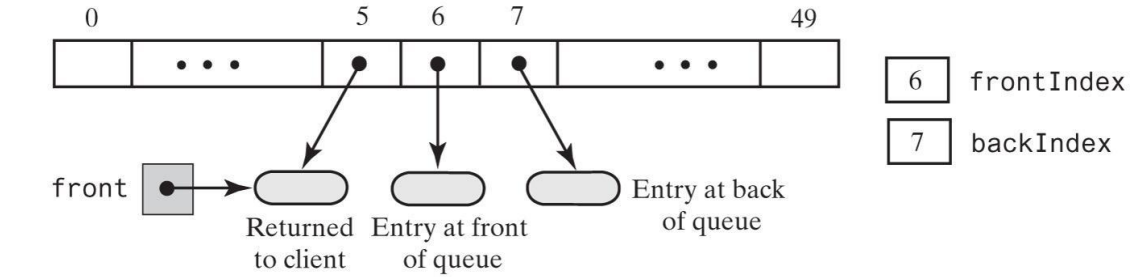
Full: frontIndex = (backIndex + 2) % queue.length

Circular Array with One Unused Element

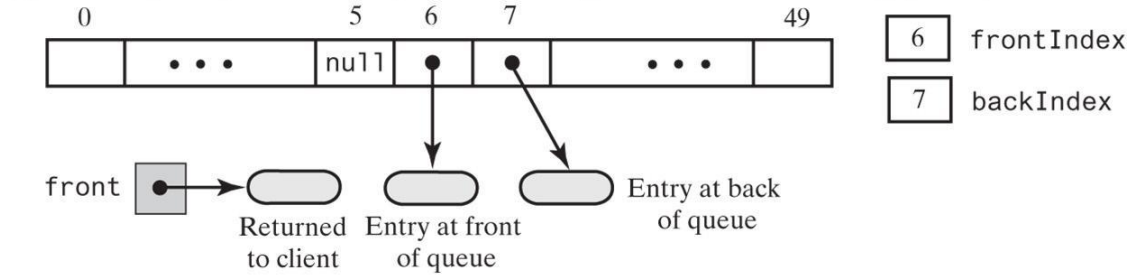
(a) Initially



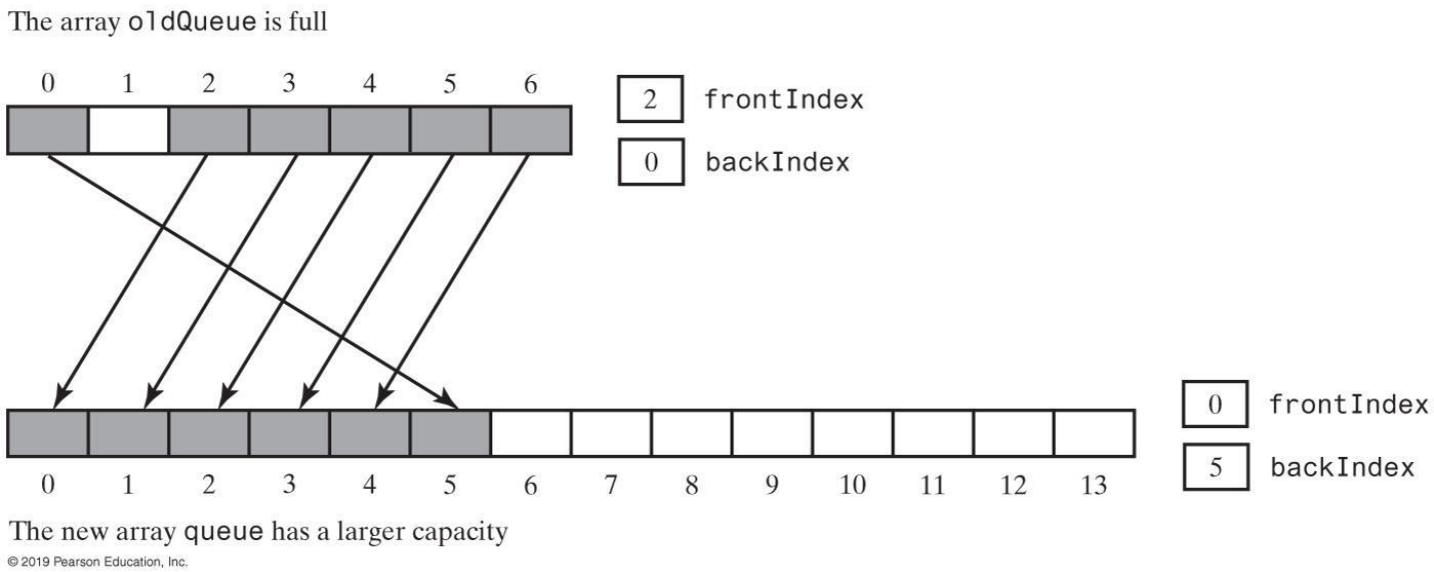
(b) After dequeuing the front entry by incrementing `frontIndex`



(c) After dequeuing the front entry by incrementing `frontIndex` and setting `queue[frontIndex]` to `null`



Circular Array with One Unused Element

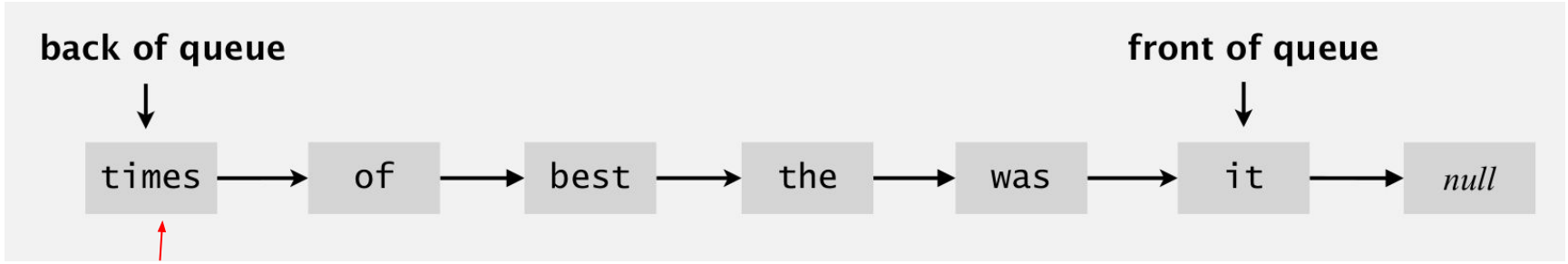



```
for (int index = 0; index <oldSize-1; index ++){
    queue[index]=oldQueue[frontIndex];
    frontIndex = (frontIndex + 1)% oldSize;}
```

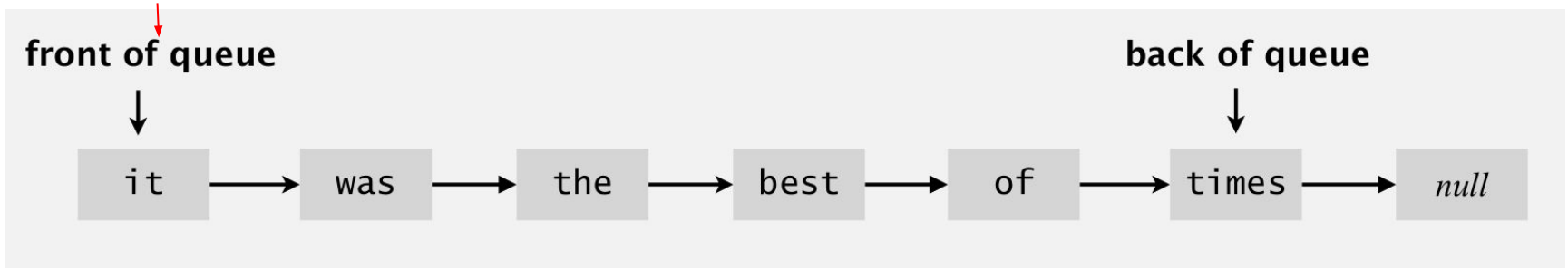
Linked Implementation of a Queue

- Enqueue "*it was the best of times*" to a queue, where are the front and back of the queue? How do you like them to be stored?

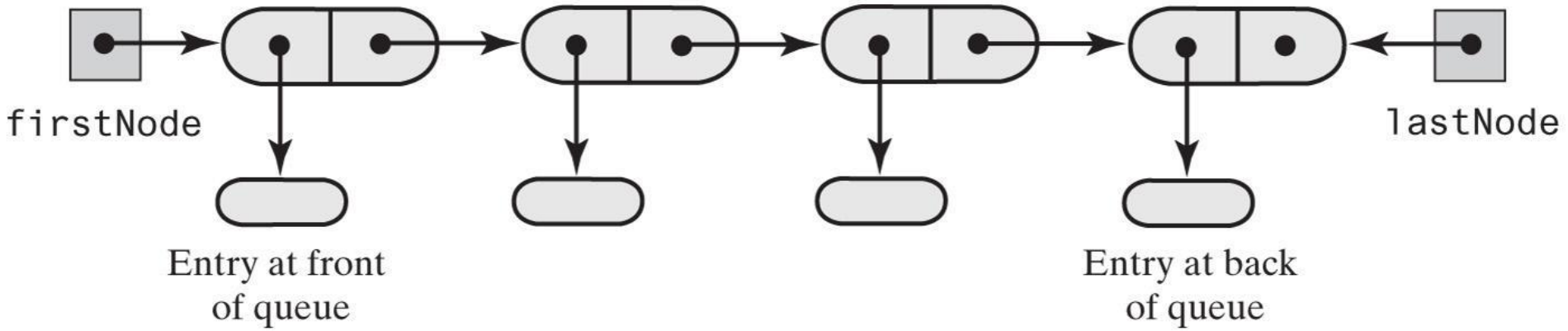
A



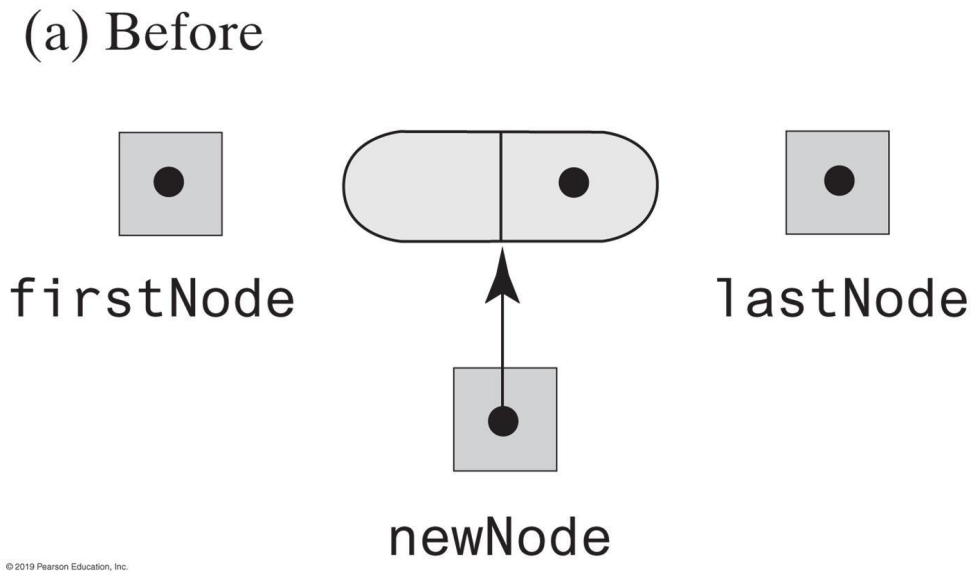
B

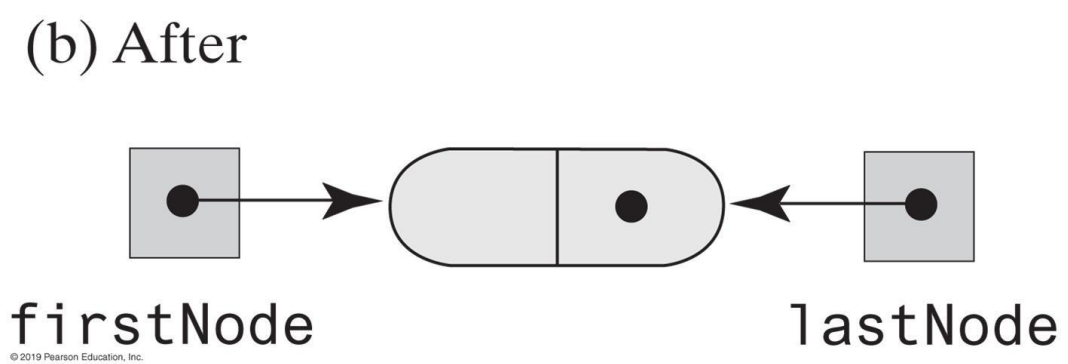


Linked Implementation of a Queue



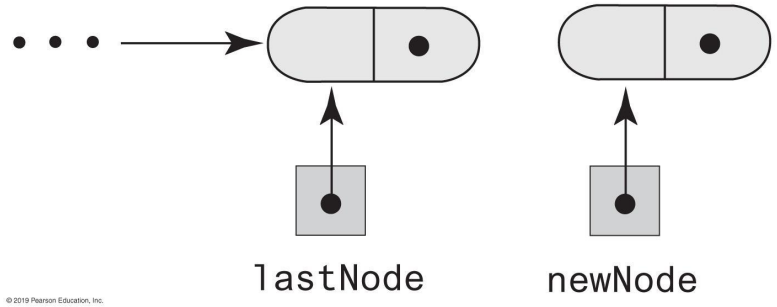
Linked Implementation of a Queue



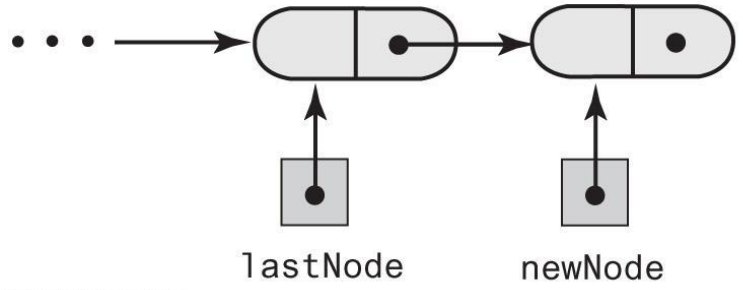


Linked Implementation of a Queue

(a) Before the addition

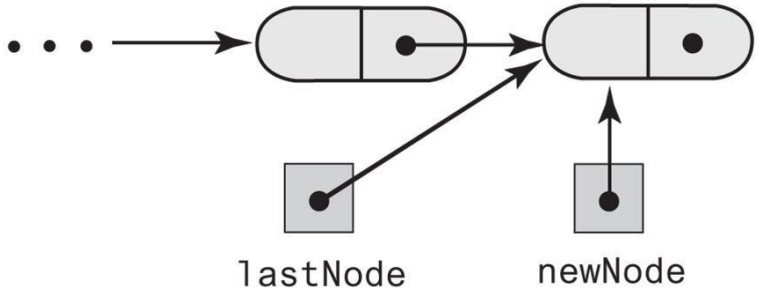


(b) During the addition



After executing
`lastNode.setNextNode(newNode);`

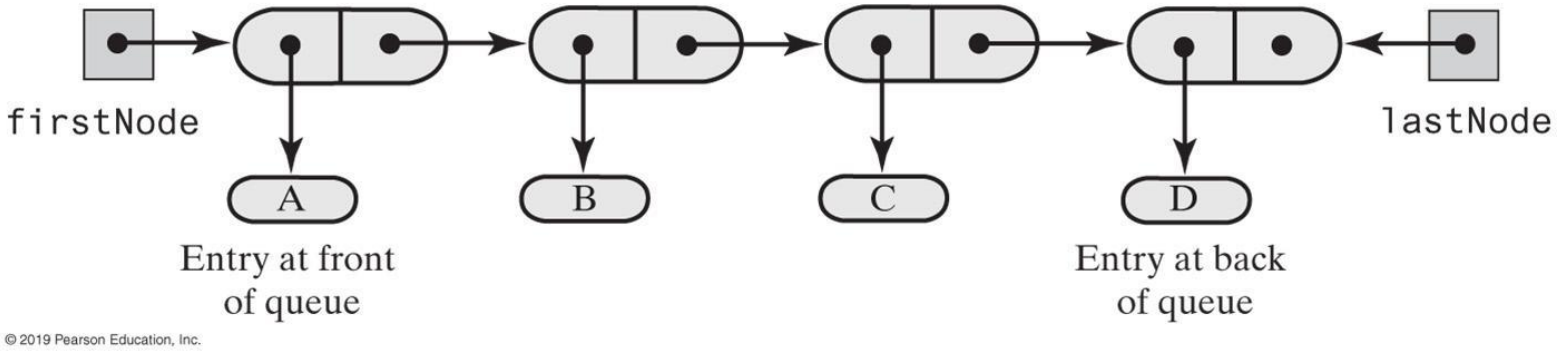
(c) After the addition



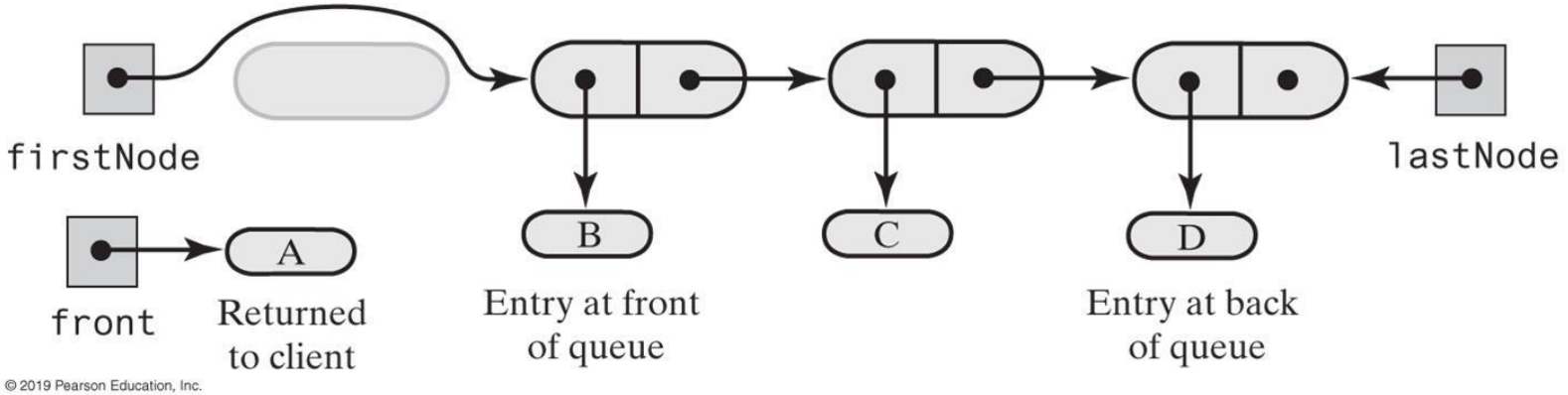
After executing
`lastNode = newNode;`

Linked Implementation of a Queue

(a) A queue of more than one entry

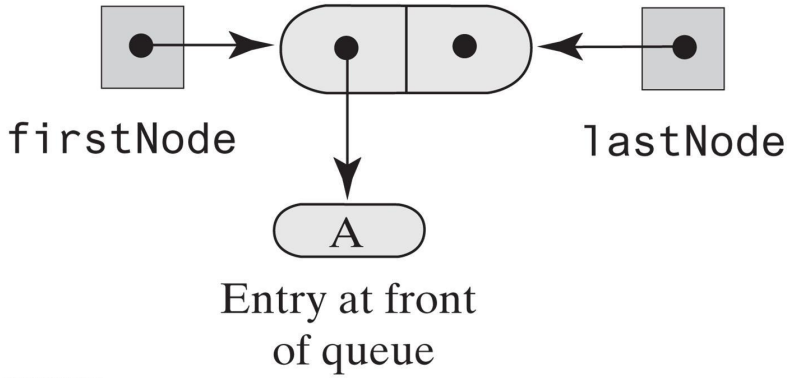


(b) After removing the entry at the queue's front

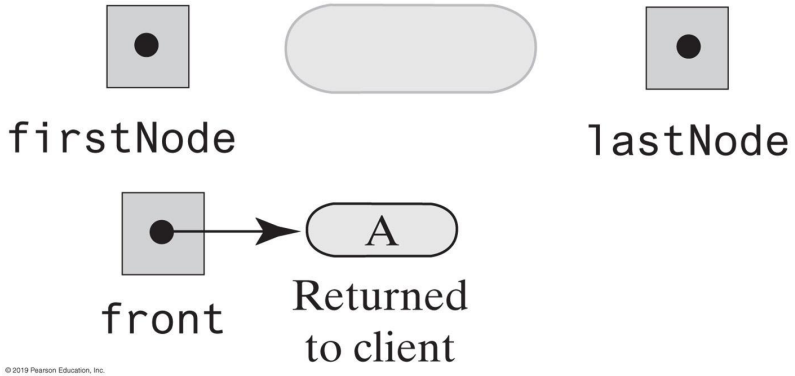


Linked Implementation of a Queue

(a) A queue of one entry



(b) After removing the only entry



Linked Implementation of a Queue

An outline of a linked implementation of the ADT queue

```
/** A class that implements a queue of objects by using
   a chain of linked nodes that has both head and tail references. */
public final class LinkedQueue<T> implements QueueInterface<T>
{
      private Node firstNode; // References node at front of queue
      private Node lastNode;  // References node at back of queue

      public LinkedQueue()
      {
            firstNode = null;
            lastNode = null;
      } // end default constructor

    //  < Implementations of the queue operations go here. >
      private class Node
      {   //  < Implementation of the inner class Node goes here. >

      } // end Node
} // end LinkedQueue
```

Linked Implementation of a Queue

```
public void enqueue(T newEntry)
{
   Node newNode = new Node(newEntry,
null);

   if (isEmpty())
      firstNode = newNode;
   else
      lastNode.setNextNode(newNode);

   lastNode = newNode;
} // end enqueue
```

```
public T getFront()
{
   if (isEmpty())
      throw new
EmptyQueueException();
   else
      return firstNode.getData();
} // end getFront
```

enqueue is O(1)

Linked Implementation of a Queue

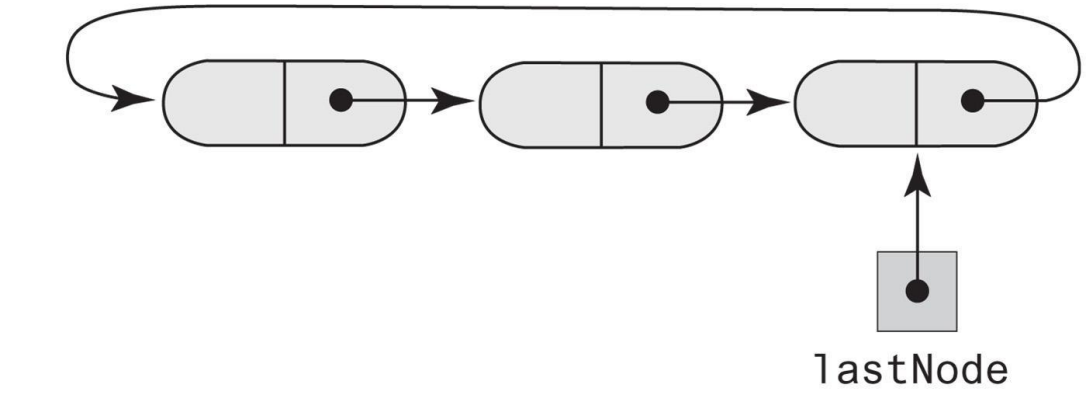
```
public T dequeue()
{
   T front = getFront();  // Might throw
EmptyQueueException
   // Assertion: firstNode != null
   firstNode.setData(null);
   firstNode = firstNode.getNextNode();

   if (firstNode == null)
      lastNode = null;

   return front;
} // end dequeue
```

dequeue is O(1)

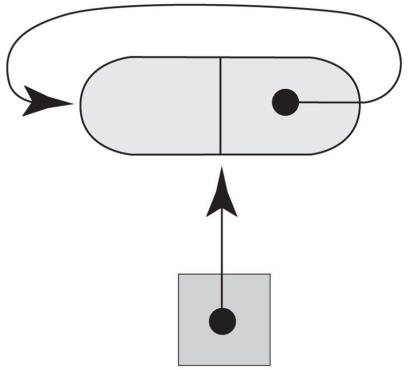
```
public boolean isEmpty()
{
   return (firstNode == null) && (lastNode ==
null);
} // end isEmpty

public void clear()
{
   firstNode = null;
   lastNode = null;
} // end clear
```

Circular Linked Implementations of a Queue

(a) A multinode chain



(b) A one-node chain

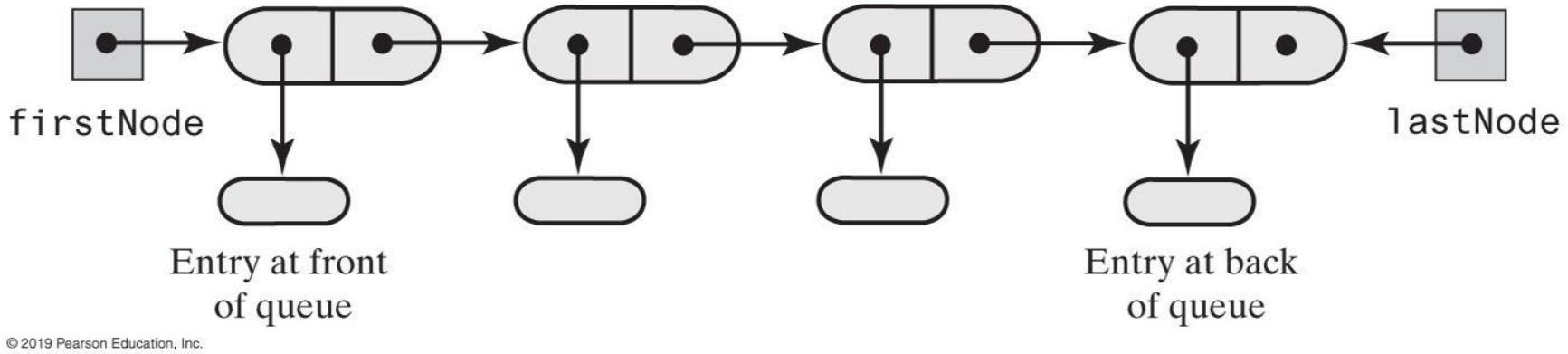


(c) An empty chain

What is the main difference compared to linear linked list?

Only one reference to the last node is required!

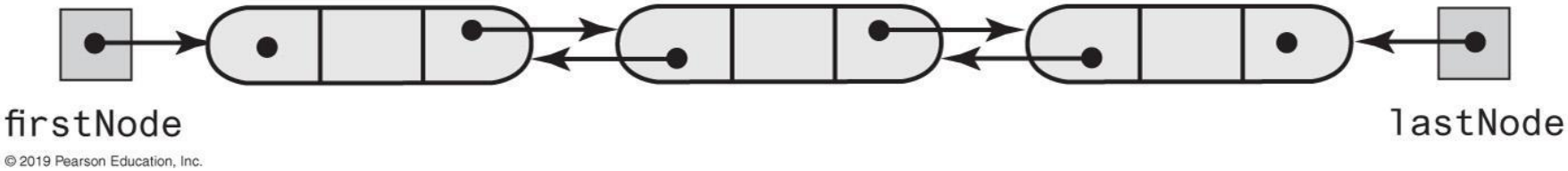
Implementation of a Deque



What would be the problem if we just use this singly linked chain to implement a deque?

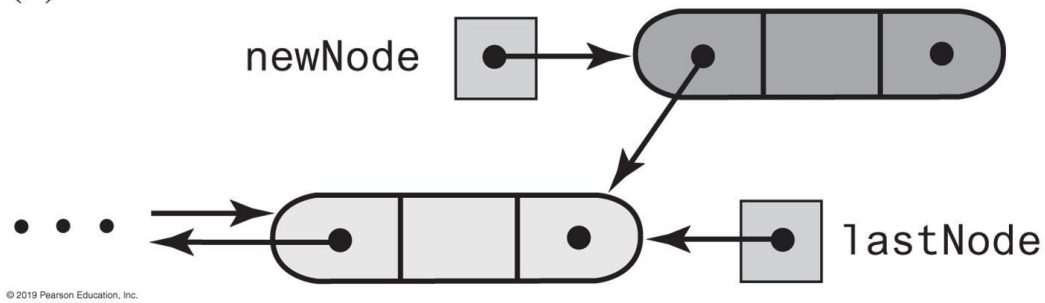
Cannot move backward!

Doubly Linked Implementation of a Deque

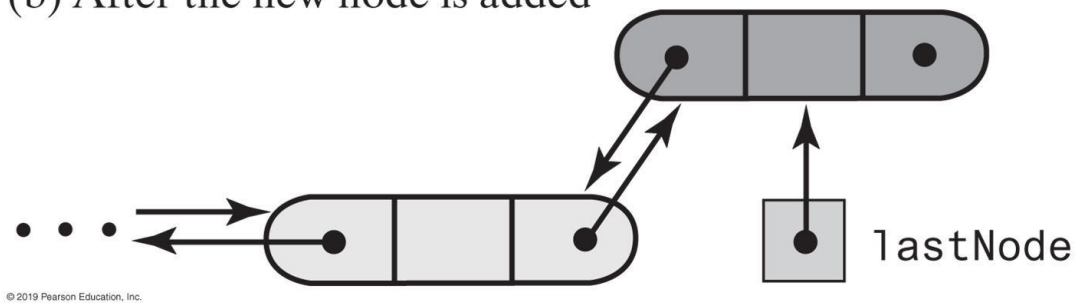


Doubly Linked Implementation of a Deque

(a) After the new node is allocated

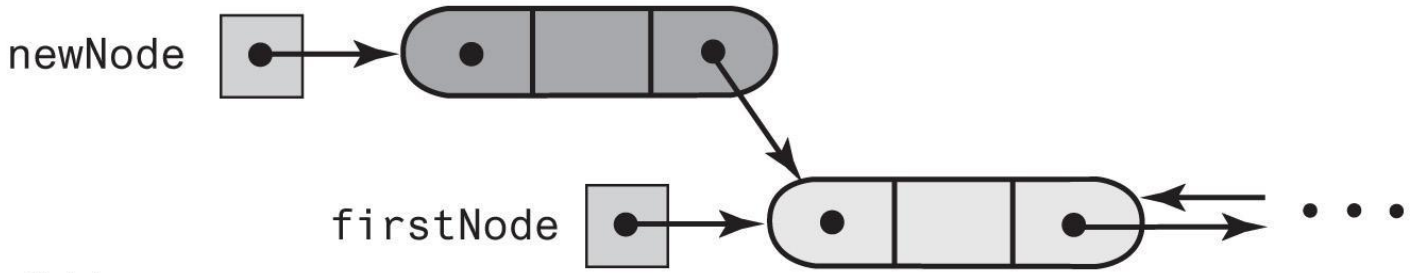


(b) After the new node is added



Doubly Linked Implementation of a Deque

(a) After the new node is allocated



(b) After the new node is added to the front

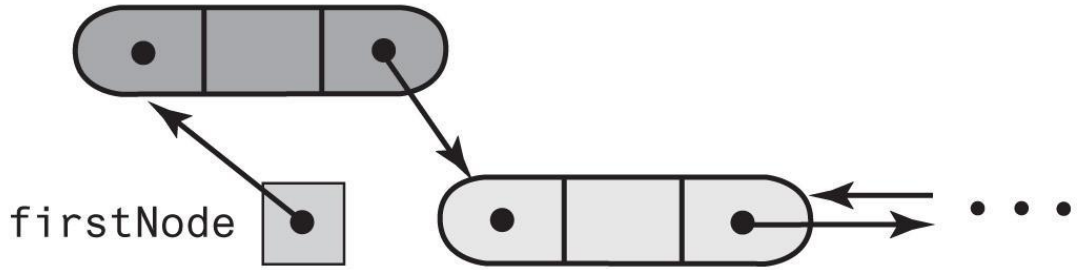
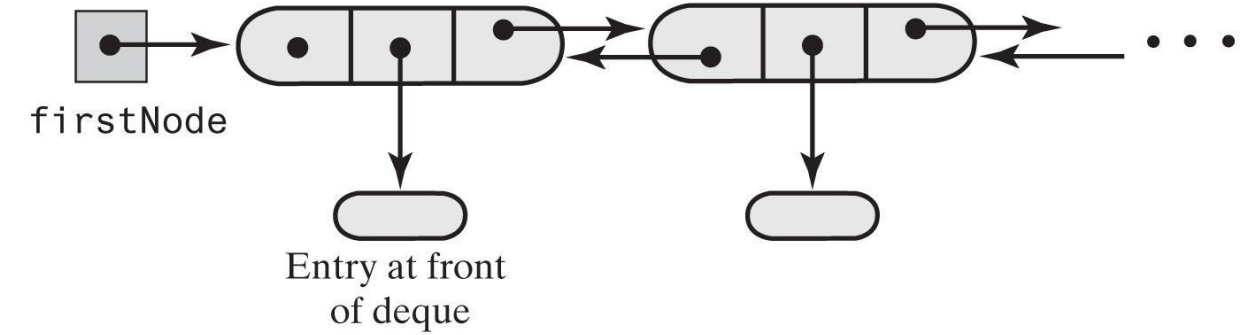


FIGURE 8-19 Adding to the front of a nonempty deque

Doubly Linked Implementation of a Deque

(a) A deque containing at least two entries



(b) After removing the first node and returning a reference to its data

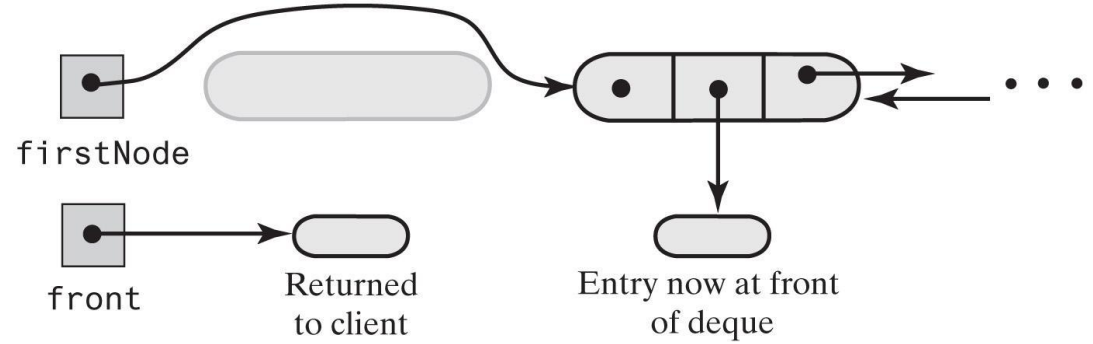
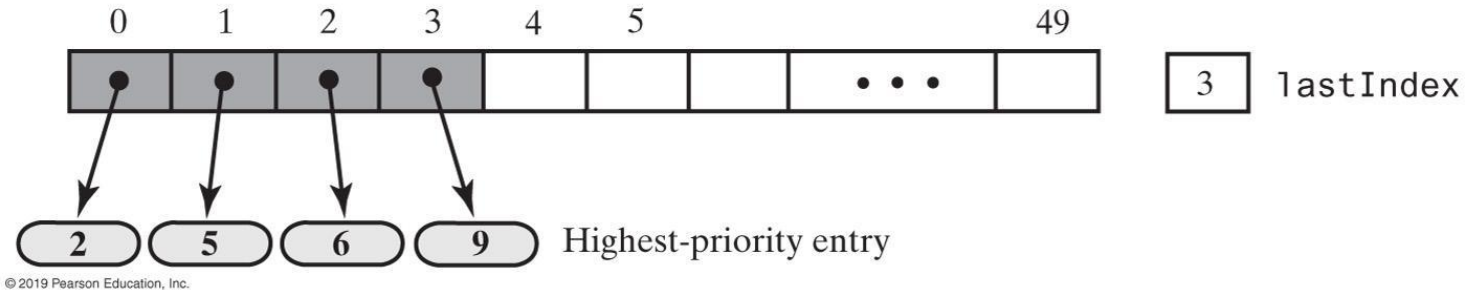


FIGURE 8-20 Removing the front of a deque containing at least two entries

Possible Implementations of a Priority Queue

(a) Array based



(b) Link based

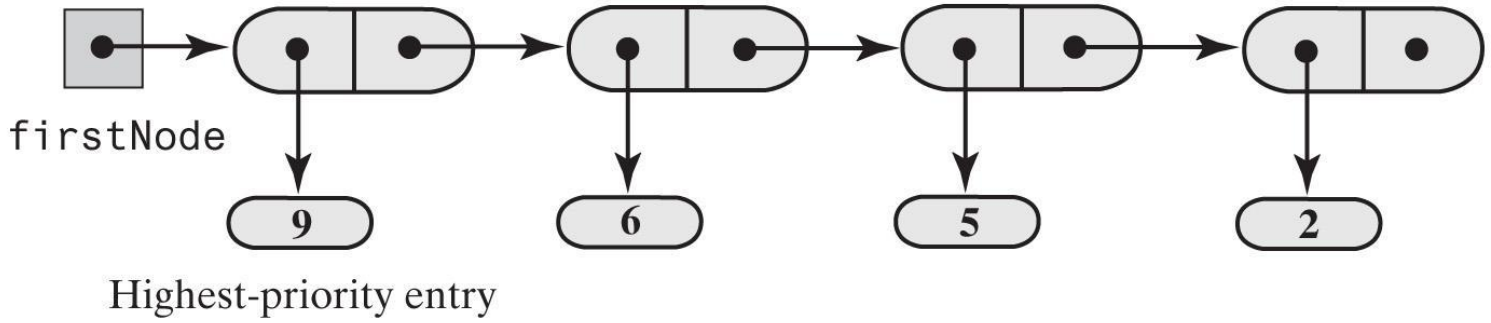


FIGURE 8-21 Two possible implementations of a priority queue